

40.016 The Analytics Edge

Clustering and Recommendation Systems

Yihan Du

Engineering Systems and Design (ESD)
Singapore University of Technology and Design (SUTD)

Summer 2026

Recommendation Systems?

- Personalize the user experience for online applications.
- Leverage data on items and customers (e.g., likes, purchase history).
- A key challenge: they must be fast and accurate.
- Common underlying analytics: clustering, collaborative filtering.

Netflix Cinematch

Netflix Cinematch

- Netflix's original movie recommendation system that predicts how users would rate movies.
- Launched in 2000.
- Accurate within half a star (five-star system) 75% of the time.
- Need for a more advanced recommendation system.

Netflix Prize

Netflix Prize

A machine learning competition launched by Netflix in 2006 to improve the accuracy of its movie recommendation system, Cinematch.

Key aspects of the competition

- 1 million USD prize for improving Cinematch by more than 10% on the test dataset.
- Training dataset: 100,480,507 ratings that 480,189 users gave to $\sim 18,000$ movies.
- Test dataset: 2,817,131 data points, used for the public and private leaderboard.
- Test predictions were scored compared to the true ratings.

Netflix Prize

Timeline

- October 2006: the competition was launched.
- November 2007: BellKor wins the 50,000 USD progress prize with a 8.43% improvement over Cinematch.
- June 2009: A team named BellKor Pragmatic Chaos achieves 10.05% improvement.
- September 2009: Team BellKor Pragmatic Chaos officially wins the competition.

A few words from the winner: [From the Labs: Winning the Netflix Prize.](#)

Agenda

- Clustering
 - Hierarchical Clustering
 - K-means Clustering
- Recommendation systems
 - Collaborative Filtering
- R Implementation

Clustering

Clustering

Goal. Partition observations into distinct groups so that

- observations within each group are quite similar to each other;
- observations in different groups are quite different from each other

This is an unsupervised problem, because we are trying to discover clusters (or, in general, a structure) within a dataset. We are not trying to predict the outcome of a given variable using training data with labels.

Clustering

Common methods

- Hierarchical clustering: we do not know in advance how many clusters we want (bottom-up).
- K-means clustering: we partition the observations into a pre-specified number of clusters.

Hierarchical Clustering

Idea

- Start with observation as its own cluster
- Merge the closest clusters step by step
- Repeat until all points form one big cluster

Hierarchical Clustering

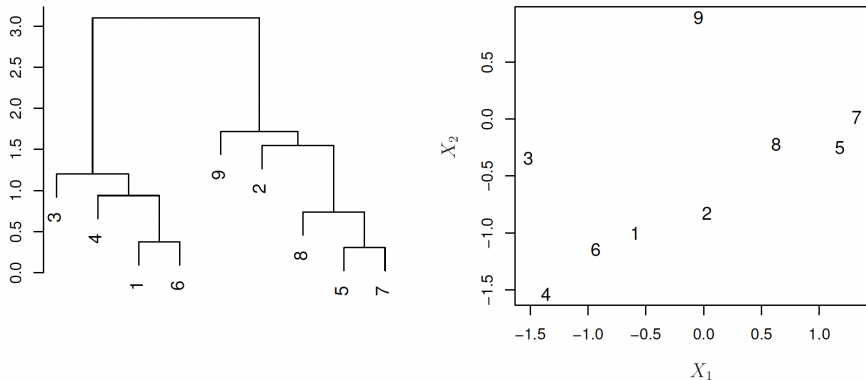


Figure: Hierarchical clustering example (Source: James et al., 2014)

Hierarchical Clustering

Algorithm

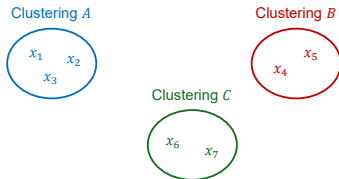
- (1) Begin with n observations and a measure (such as Euclidean distance) of all the $n(n-1)/2$ pairwise dissimilarities. Treat each observation as its own cluster.
- (2) For $i = n, n-1, \dots, 2$:
 - Examine all pairwise inter-cluster dissimilarities among the i clusters and identify the pair of clusters that are least dissimilar (that is, most similar). Merge these two clusters.
 - Compute the new pairwise inter-cluster dissimilarities among the $i-1$ remaining clusters.

Hierarchical Clustering

When a cluster contains multiple observations, how do we define the dissimilarity (distance) between two clusters?

Common types of **linkage**:

- Complete: maximum pairwise distance between observations in the two clusters
- Single: minimum pairwise distance between observations in the two clusters
- Average: average pairwise distance between observations in the two clusters
- Centroid: distance between the cluster centroids
 - Cluster centroid: the mean of the observations contained in this cluster
- ward.D2: merge the two clusters that yield the smallest increase in total within-cluster variance



R Implementation

We use `hclust(distance matrix, method)`

K-Means Clustering

Let $C_1, \dots, C_K$ denote the sets containing the indices of the observations in each cluster. These sets satisfy two properties:

- Each observation belongs to at least one of the K clusters:
$$C_1 \cup C_2 \cup \dots \cup C_K = \{1, \dots, n\}$$
- Clusters are non-overlapping: $C_k \cap C_{k'} = \emptyset$ for all $k \neq k'$

K-Means Clustering

Idea

A good clustering is one for which the within-cluster variation is as small as possible.

We want to find a clustering $C_1, \dots, C_K$ that minimizes the sum of within-cluster variations over all clusters:

$$\min_{C_1, \dots, C_K} \sum_{k=1}^K V(C_k).$$

K-Means Clustering

Suppose that an observation has p features. Denote the i -th observation in a cluster C_k by $x_i = [x_{i1}, \dots, x_{ip}]^\top \in \mathbb{R}^p$.

Denote the average observation in this cluster by $\bar{x} = [\bar{x}_1, \dots, \bar{x}_p]^\top \in \mathbb{R}^p$, where the j -th feature of $\bar{x}$ is $\bar{x}_j = \frac{1}{|C_k|} \sum_{i \in C_k} x_{ij}$ for $j = 1, \dots, p$.

We quantify the within-cluster variation by

$$V(C_k) = \frac{1}{|C_k|} \sum_{i, i' \in C_k} \sum_{j=1}^p (x_{ij} - x_{i'j})^2,$$

which is equivalent to $\frac{1}{2}$ times the sum of the Euclidean distances between each observation x_i and the average observation $\bar{x}$ in this cluster:

$$V(C_k) = \frac{1}{2} \sum_{i \in C_k} \sum_{j=1}^p (x_{ij} - \bar{x}_j)^2.$$

K-Means Clustering

Putting the previous equations together, we get

$$\min_{C_1, \dots, C_K} \sum_{k=1}^K \frac{1}{|C_k|} \sum_{i, i' \in C_k} \sum_{j=1}^p (x_{ij} - x_{i'j})^2.$$

This is the optimization problem we want to solve. The K-means algorithm uses an heuristic search process to solve it.

K-Means Clustering

Algorithm

Step 1. Randomly assign a number, from 1 to K , to each of the observations (initial cluster assignment).

Step 2. Iterate until the cluster assignment does not change:

- (a) For each of the K clusters, compute the cluster centroid. For the k -th cluster, the **centroid** is the average observation $\bar{x}$ in this cluster (for each feature, compute the average value among all observations in this cluster):

$$\bar{x} = \left[\frac{1}{|C_k|} \sum_{i \in C_k} x_{i1}, \dots, \frac{1}{|C_k|} \sum_{i \in C_k} x_{ip} \right]^T \in \mathbb{R}^p$$

- (b) Assign each observation to the cluster whose centroid is closest.

K-Means Clustering

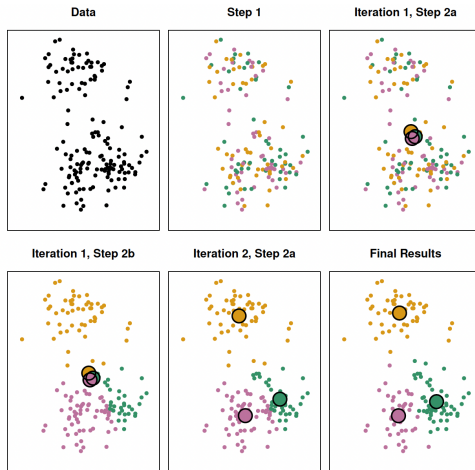


Figure: K-Means clustering Example with $K = 3$, $n = 100$ and $p = 2$ (Source: James et al., 2014)

K-Means Clustering

Some considerations

- The results will depend on the random initialization.
- It is good practice to run the algorithm multiple times (with different random initializations).
- We pick the clustering result that minimizes the value of the objective function (the total within-cluster variation).

Final Remarks on Clustering Methods

Use these questions to guide your iterative testing and model refinement in practice.

- Should the observations or features first be standardized in some way?
It is good practice to perform standardization.
- For hierarchical clustering:
 - What type of linkage should be used?
 - How many clusters should we cut the clustering tree into?
- For K-means clustering:
 - How many clusters should we use?

R Implementation

`kmeans(Data, centers = number of clusters, nstart = number of random initials)`

Recommendation systems

Recommendation Systems

Collaborative filtering

Recommendations are based on past user interactions and similarities between users.

Each user is represented by a vector where the i -th entry is the rating that this user gives to the i -th item. This vector will typically have many empty entries (only a small fraction of the items is ranked or purchased).

Recommendation Systems

Content filtering

Recommendations are made based on attributes of items.

Each item is represented by a set of attributes (e.g., genre of movie, keywords, or webpage).

Recommendation Systems

Hybrid

This is a combination of both collaborative and content filtering.

Collaborative Filtering

Baseline model

Predict the rating based on the average rating that this item receives:

$$b_{ui} = \bar{r}_i.$$

Predict the rating based on the average rating that this user gives:

$$b_{ui} = \bar{r}_u.$$

Collaborative Filtering

User-based collaborative filtering

- (1) Identify the users whose rating patterns are similar to those of the current user (i.e., neighbors).
- (2) Use the neighbors' ratings on this item to predict the rating that the current user will give to this item.

Collaborative Filtering

User-based collaborative filtering

To measure the similarity between users u and v , we can use the Pearson correlation coefficient

$$S_{uv} = \frac{\sum_{i \in I_u \cap I_v} (r_{ui} - \bar{r}_u)(r_{vi} - \bar{r}_v)}{\sqrt{\sum_{i \in I_u \cap I_v} (r_{ui} - \bar{r}_u)^2 \sum_{i \in I_u \cap I_v} (r_{vi} - \bar{r}_v)^2}},$$

where S_{uv} is the similarity between users u and v , I_u and I_v are the items rated by users u and v , $\bar{r}_u$ and $\bar{r}_v$ are the average ratings of users u and v .

This similarity metric was adopted by Netflix.

Collaborative Filtering

User-based collaborative filtering

To predict the rating, we can use the information on the neighbors of user u :

$$p_{ui} = \frac{\sum_{v \in N_u} r_{vi}}{|N_u|},$$

where N_u is the set of neighbours of user u .

The neighbors are typically chosen as the users whose rating patterns are most similar to user u , based on a similarity measure such as Pearson correlation coefficients.

In practice, we can choose the users v with the highest several Pearson correlation coefficients S_{uv} as the neighbors N_u for user u .

Collaborative Filtering

User-based collaborative filtering

A slightly more complex model:

$$p_{ui} = \frac{\sum_{v \in N_u} S_{uv} r_{vi}}{\sum_{v \in N_u} S_{uv}}$$

Collaborative Filtering

User-based collaborative filtering

The performance can be improved by normalizing the ratings to compensate for user's differences on rating scales:

$$p_{ui} = \bar{r}_u + \frac{\sum_{v \in N_u} S_{uv} (r_{vi} - \bar{r}_v)}{\sum_{v \in N_u} S_{uv}}$$

Collaborative Filtering

Assessing performance

We can use the root mean squared error (RMSE)

$$RMSE_u = \sqrt{\sum_i \frac{(p_{ui} - r_{ui})^2}{n}},$$

where p_{ui} is the predicted rating that user u gives to item i , r_{ui} is the real rating, and n is the number of predicted ratings.

R Implementation

`ratings.csv` dataset

R Implementation

Data preparation

R Implementation

Baseline model 1

R Implementation

Baseline model 2

R Implementation

User-based model

R Implementation

User-based model

Reference

Clustering and Recommendation Systems

- Teaching notes.
- James et al. (2014) An Introduction to Statistical Learning with Applications in R, Springer, 2014. Chapter 10.3.

Netflix

- Koren Y. "The Bellkor solution to the Netflix grand prize." Netflix prize documentation 81.2009 (2009): 1-10.
- From the Labs: Winning the Netflix Prize
- Why Netflix's Algorithm Is So Binge-Worthy
- Why your Netflix thumbnails don't look like mine